Document Name: Custom Millisecond Timer — System Design Document

Document Reference: CMPE311-PROJECT-MYMILLI-SDD.1.2

Date of Publication: 10/15/2026

LEAD ENGINEER: Nyi Thu Kha, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION:

An embedded system based on the ELEGOO Uno R3 that allows a user to set the blink intervals of two LEDs independently through the Arduino IDE serial monitor, using a round-robin cyclic executive task manager and a custom millisecond timer.

DESCRIPTION:

The system uses an Arduino-compatible development board, two LEDs on digital pins 2 and 3, and a bare-metal, non-blocking software architecture organized as a round-robin cyclic executive with a function pointer table. The Arduino millis() function has been replaced by a custom mymilli() function. This function is backed by a custom Timer0 CTC ISR and locally defined register constants. The user enters which LED to change and the desired blink interval in milliseconds. Each LED then blinks at its own independent rate.

RESULT SUMMARY:

The system was designed, built, and tested. It successfully accepted serial commands, updated each LED interval independently, and maintained constant blinking using the custom mymilli() timer. All required tests, including those for the cyclic executive and the custom timer, were satisfied.

REFERENCES

1. Nyi Thu Kha, Cyclic Executive Task Manager — System Design Document, CMPE311-PROJECT-CE-SDD1.1, 9/29/2026
2. Nyi Thu Kha, Async LED Blink Controller — System Design Document, CMPE 311-ASYNC-LED-NYI-92626
3. How to do Multitasking with the Arduino
4. The Arduino Handbook – Learn Microcontrollers for Embedded Systems
5. sec04b-engineeringProcess_pt1.pdf – Engineering Process Part 1
6. sec04b-engineeringProcess_pt2.pdf – Engineering Process Part 2
7. CMPE310 asynchronous Arduino lab
8. Arduino IDE 2.3.3 or later documentation
9. ATmega328P datasheet

DEFINITIONS

The User	The person operating (not programming) the embedded system
The System	The embedded system being operated by *The User*
The Customer	The person(s) paying for the embedded system being designed and built
The Developer	The person(s) designing and building the *System*
The Evaluator	The person(s) that determine whether or not *The System* satisfies *The Customer-requirements*
The Customer Requirements	The requirements defined by *The Customer* as satisfying *The Contract*
The HL-requirements	The System's high-level technical requirements derived from *The Customer-requirements*
The Requirements	context dependent; e.g. may refer to the *HL-requirements* or *Customer requirements*
The Educational-constraints	The requirements imposed by the instructor unrelated to the embedded system that allow *The System* to be evaluated.
The Company	The organization on *The Customer* has contracted with to build *The System*
The Contract	The business document that legally binds *The Company* to provide some service or product to *The Customer*
serial-monitor	The serial port used by the Arduino IDE to communicate with *The User*
The Reference-platform	The configuration on of *The System* used by *The Developer* to test and validate *The System*. For this class, *The System* is the Arduino compatible ELEGOO Uno R3 development board
Asynchronous	Serial communication and LED timing that do not block or wait on each other

Non-Blocking	Code that does not use delay() to pause execution
Millis()	Arduino function returning milliseconds since program start, used for timing
Blink Interval	Full on/off period in milliseconds. On-time and off-time are each half the interval
Duty Cycle	Percentage of time the LED is on; this design used 50%
Cyclic Executive	A scheduling approach in which the main loop dispatches a fixed set of tasks in a repeating order
Round-Robin	A cyclic executive policy in which tasks are visited one at a time in a fixed order and the sequence wraps back to the first task
Function Pointer Table	An array of function pointers used to dispatch tasks by index
Task	A function that performs one unit of work and returns promptly without blocking
Dispatcher	The portion of loop() that selects and calls the next task
ISR	Interrupt Service Routine. A function that runs in response to a hardware interrupt.
CTC Mode	Clear Timer on Compare Match. A timer mode where the counter resets to zero when it reaches a programmed compare value.
Prescaler	A divider that slows the timer clock relative to the CPU clock.
Atomic Access	Reading or writing a multi-byte variable with interrupts disabled so that an ISR cannot corrupt the value mid-operation.
mymilli()	Custom replacement for Arduino's millis(), implemented by the Developer.

ACRONYMS AND ABBREVIATIONS

Arduino	an Italian open-source hardware and software company; also refers to a development board created by that company
arduino.h	header for a library of convenience functions specific to the Arduino development platform
AVR	A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology
avr.h	header including helper functions and definitions for using an AVR microcontroller
ELEGOO	A Chinese company that develops and markets 3D printers and accessories
HL	High-level
IDE	Integrated Development Environment
gcc	Front end of the GNU Compiler Collection
GitHub	A widely used distributed SVC (Software Version Control) system

LED	Light Emitting Diode
RTOS	Real-Time Operating System
ISR	Interrupt Service Routine
SVC	Software Version Control
UART	Universal Asynchronous Receiver/Trasmitter
CE	Cyclic Executive
ECN	Engineering Change Note
ECO	Engineering Change Order
ECR	Engineering Change Request
CTC	Clear Timer on Compare Match
PWM	Pulse Width Modulation

CONVENTIONS

Must, **shall** or **will**	- your design must satisfy the requirement
May	- your design may satisfy the requirement but doesn't have to
Informative	- the intent of the following descrip2on is to make the requirement more understandable

All customer requirements are started with **C.#**

All high-level technical requirements are started with **HL.#**

All tes2ng/valida2on requirements ar5e started with **T.#**

CUSTOMER REQUIREMENTS

C1.	The User will be able to choose the blink rate of the LED with a button.
C2.	The User will choose between 4 rates plus off.
C3.	The blink rates must be set by an operator prior to the system being started.
C4.	Permitted blink rates must be perceivable by the viewer.
C5.	The minimum blink rate must be no slower than once every 5 seconds.
C6.	The System must run upon an Arduino Uno R3 compatible board.

PART I – HIGH-LEVEL TECHNICAL REQUIREMENTS DERIVED FROM C1-C6

HL.1	The System must use an Arduino Uno R3 compatible development board.
HL.2	The System must use at least one LED.
HL.3	The System must use at least one momentary push-button.
HL.4	The System must provide five user-selectable modes: OFF four blink rates.
HL.5	The four blink rates must be nominally 0.2 Hz, 1 Hz, 2 Hz, and 5 Hz.
HL.6	The User must select the mode using the pushbutton before normal operation begins.
HL.7	The selected mode must be stored and used after the System starts.
HL.8	The slowest blink rate must have a period no greater than 5.0 seconds.
HL.9	Each blink rate must produce alternating ON/OFF states with a 50% duty cycle.
HL.10	The System must not require a serial monitor or host computer for mode selection.
HL.11	The System must run standalone once programmed and powered.

Note: These Part I HLs are not used in Parts II and III.

PART II/III – HIGH-LEVEL TECHNICAL REQUIREMENTS GIVEN IN TABLE 3

Table 1. High-Level Requirements

HL.1	The System must use at least 2 LEDs.
HL.2	The System must use a standard Arduino compatible development board, like the provided ELEGOO Uno R3.
HL.3	The System must communicate with The User via the Arduino IDE serial-monitor port.
HL.4	Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.
HL.5	The User must set the blink interval of the LEDs in msec.
HL.6	The blink rate of each LED must be constant unless changed by The User.

EDUCATIONAL CONSTRAINTS

Table 2. Educational Constraints

EC.1	The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System.
EC.2	The Developer must have any software used in The System hosted on a GitHub repository. This software does not need to include libraries, macros, or other code not developed by The Developer.
EC.2.1	The Evaluator must be able to download from the repository any software, documentation, and other material needed for their evaluation, cross compile any software, and run that software on The Reference Platform.
EC.3	LEDs must be connected to Arduino digital Pins 2 and 3.
EC.4	The Evaluator must be able to successfully have The System perform the dialog in Table 5.
EC.5	The Developer must do bare metal programming, i.e., they cannot use any 3rd party multi-tasking libraries or programs such as FreeRTOS.
EC.6	The Developer must implement a cyclic executive task manager based upon a function pointer table to dispatch the asynchronous tasks created in PROJECT-ASYNC.
EC.6.1	The system testing and validation requirements must contain those defined in PROJECT-ASYNC.
EC.6.1.1	The testing and validation requirements must be extended to contain any additional tests necessary to properly evaluate and demonstrate the function of the modified system.
EC.6.2	The final report must be a new revision of the formal design document produced in PROJECT-ASYNC, and must note any changes made to the design.

EC.7	The Developer must replace the millis() function provided by arduino.h with a custom function named mymilli(). The custom function may extend the capability of millis() beyond simple millisecond counting.
EC.7.1	The Developer must write a custom PWM counter ISR to provide the timing mechanism underlying mymilli().
EC.7.2	The Developer must define custom register constants that would normally be defined in avr.h. These constants do not need to use the same names as avr.h.
EC.7.3	The system testing and validation requirements must contain those defined in PROJECT-CE.
EC.7.3.1	The testing and validation requirements must be extended to include any additional tests necessary to properly evaluate and demonstrate the function of the modified system.
EC.7.4	The final report must be a new revision of the formal design document produced in PROJECT-CE, and must note any changes made to the design.
EC.7.5	The Developer must provide a properly formatted ECN notifying other teams of the changes.

DEVELOPMENT PLATFORM

1. ELEGOO Uno R3 development board or compatible Arduino Uno R3
2. Arduino IDE 2.3.3 or better
3. gcc toolchain used by Arduino IDE for AVR ATmega328p
4. 2 × LEDs
5. 2 × 200 $\Omega \frac{3}{4}\ W$ resistors
6. 1 × 10 $k\Omega \frac{3}{4} W$ resistors
7. Breadboard and jumper wires
8. USB 2.0 female type B to female USB 2.0 type A cable
9. Laptop/computer running Arduino IDE
10. GitHub repository for source and documentation

ADDITIONAL DESIGN CONSIDERATIONS

1. The hardware design is identical to that of PROJECT-ASYNC. See [1] SYSTEM DESIGN section.
2. The software design uses a round-robin cyclic executive with a function pointer table.
3. The software is organized into three non-blocking tasks: taskSerial, taskLed1, and taskLed2.
4. The dispatcher in loop() calls exactly one task per iteration and advances the task index modulo the number of tasks.
5. millis() is used for all LED timing.
6. Shared state is accessible to all tasks at file scope.
7. No delay() is used in the main loop.
8. No FreeRTOS or third-party multitasking library is used.
9. The serial monitor baud rate is 9600.
10. Arduino's millis() is replaced by a custom mymilli() function.
11. mymilli() is driven by a custom Timer0 ISR configured in CTC mode with OCR0A = 249, producing exactly one interrupt per millisecond.
12. Register access uses locally defined constants (TIMER0_CTRL_A, TIMER0_CTRL_B, TIMER0_COUNT, TIMER0_COMPARE_A, TIMER0_INT_MASK) instead of avr.h names.

13. The mymilli() read is atomic: interrupts are disabled during the read and the previous interrupt state is restored afterward.

14. Because Timer0 is under Developer control, Arduino's millis() and delay() no longer function correctly and are not used anywhere in the software.

SCHEMATIC DESCRIPTION

Identical to PROJECT-ASYNC. See [1] SCHEMATIC DESCRIPTION section.

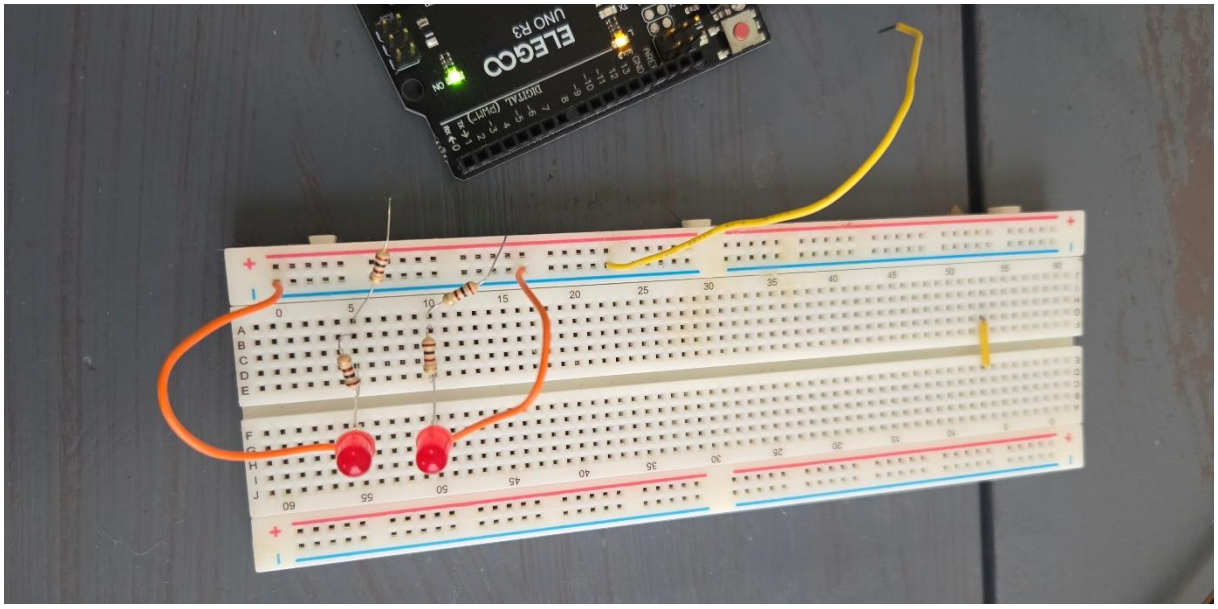


Figure 1 (Photo of physical Circuit). *Note: 2 100 Ω resistors were used for 200 Ω resistors*

SYSTEM DESIGN

1. **Overview**
   The hardware design is identical to that of PROJECT-ASYNC. See [1] ADDITIONAL DESIGN CONSIDERATIONS and SCHEMATIC DESCRIPTION sections.

   The software design has changed. PROJECT-ASYNC used a single flat loop() that handled serial parsing and both LED toggles inline. PROJECT-CE restructures the same functionality into a round-robin cyclic executive driven by a function pointer table. The behavior visible to The User is unchanged.

2. **Software Architecture**
   The software is organized into three non-blocking tasks:

Task	Responsibility
taskSerial()	Reads and parses serial input; updates the selected LED's interval.

taskLed1()	Toggles LED1 when its half-interval has elapsed.
taskLed2()	Toggles LED2 when its half-interval has elapsed.

Each task returns promptly and never blocks. No task uses delay(), waits on I/O, or spins on a condition.

3. **Cyclic Executive**

The cyclic executive is implemented in loop():

```
// Cyclic executive round-robin dispatch via the function pointer table
void loop() {
    static int currentTask = 0;
    taskTable[currentTask]();   // task dispatch
    currentTask = (currentTask + 1) % NUM_TASKS; // for round-robin
}
```

On each pass through loop(), exactly one task is dispatched. The index advances by one, and then wraps back to zero after the last entry. This produces round-robin scheduling where every task is visited in a fixed order, once per cycle, with no priorities and no preemption.

4. **Function Pointer Table**

```
// Function pointer table (cyclic executive)
typedef void (*TaskFunc)(void);
TaskFunc taskTable[] = {taskSerial, taskLed1, taskLed2};
const int NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]); // Make it constant
```

The table has three properties that matter for this design:

1. **Uniform signature**: Every task takes no arguments and returns void.
2. **Compile-time size**: NUM_TASKS is computed with sizeof, so adding or removing a task automatically updates the modulus.
3. **Indirection**: loop() dispatches whichever function the current index points to.

5. **Task Design**

taskSerial() drains the UART receive buffer and runs a two-state parser (WAIT_LED, WAIT_INTERVAL). It never blocks waiting for input.

taskLed1() and taskLed2() compare millis() against their own lastToggle timestamp. Because interval is the full period and each task toggles once per half-period, the resulting waveform is 50% duty cycle. LED1 and LED2 use separate intervals, state variables, and timestamps, so changing one does not affect the other.

6. **Timing Analysis**

The scheduler guarantees tasks execute in a fixed order at a rate determined by loop()'s iteration speed. Because there are no tasks that block, the maximum interval between successive executions of any given task is bounded by the sum of all task execution times. Each task's execution time is on the order of microseconds, and since the observed worst-case cycle time is far below the smallest supported interval, it can be assumed that all timing requirements are met with wide margin.

7. Changes from PROJECT-ASYNC

Aspect	PROJECT-ASYNC	PROJECT-CE
Main loop	All logic inline	Round-robin dispatcher
Task separation	None	taskSerial, taskLed1, taskLed2
Dispatch mechanism	Direct calls	Function pointer table
Scheduling	N/A	Round-robin
Hardware	Unchanged	Unchanged
User-visible behavior	Unchanged	Unchanged
Testing requirements	T.1-T.17	T.1-T.17 + T.18-T.22

See the accompanying ECN (ECN1_1) for the formal change record.

Custom Millisecond Timer

Arduino's built-in millis() is implemented on top of Timer0, which the Arduino core configures in overflow mode with a prescaler of 64. That configuration produces an overflow every 1.024 ms, and the Arduino core uses a fractional accumulator to correct the error. PROJECT-MYMILLI replaces this entire mechanism with a different implementation.

Timer0 is reconfigured in CTC (Clear Timer on Compare Match) mode. With a prescaler of 64, the timer clock runs at 16 MHz / 64 = 250 kHz, which is 250 ticks per millisecond. The compare register is set to OCR0A = 249, so the counter resets after 250 ticks. This produces exactly one compare-match interrupt per millisecond, with no fractional accumulator needed.

The ISR is minimal:

```
ISR(TIMER0_COMPA_vect) {
    my_timer0_millis++;
}
```

Each interrupt increments a 32-bit counter. Because the ISR runs every 1 ms and executes a single increment, its overhead is negligible.

mymilli() reads the counter atomically:

```
unsigned long mymilli(void) {
    unsigned long m;
    uint8_t oldSREG = SREG;
```

```
    cli();
    m = my_timer0_millis;
    SREG = oldSREG;
    Return m;
}
```

Because my_timer0_millis is four bytes and the ISR can fire at any moment, the read must be protected. The function saves the current interrupt state, disables interrupts, reads the counter, and restores the previous interrupt state. This prevents the ISR from modifying the counter mid-read and producing a corrupted value.

Register access uses locally defined constants rather than the names from avr.h. This isolates the design from any future changes to the AVR toolchain and avoids any third-party IP entanglement.

TESTING AND VALIDATION REQUIREMENTS

The Project 1 testing and validation requirements, T.0 through T.17, remain in effect. See [1] TESTING AND VALIDATION REQUIREMENTS section. In addition, PROJECT-CE adds the following:

Table 3. Testing and Validation Requirements

T.23	Verify mymilli() increments once per millisecond.
T.24	Verify mymilli() reads the 32-bit counter atomically.
T.25	Verify PROJECT-CE behavior (Table 5 dialog, independent blinking) is unchanged when using mymilli().
T.26	Verify the source contains no calls to Arduino millis() or delay().
T.27	Verify all Timer0 register accesses use Developer-defined constants, not avr.h names.

TESTING RESULTS

Table 4. Results of tests

Test performed	Results
T.0 – T.17	Satisfied (see [2] TESTING RESULTS section)
T.18 – T.22	Satisfied (see [1] TESTING RESULTS section
T.23	Satisfied
T.24	Satisfied
T.25	Satisfied
T.26	Satisfied
T.27	Satisfied

Evidence:

```
□□□□□□□□□□□□□start
t = 1000
t = 2000
t = 3000
t = 4000
t = 5000
t = 6000
t = 7000
t = 8000
t = 9000
```

Figure 3 (Serial Monitor Screenshot): *T.23 timer test output*

Video of independent blinking

GitHub repository link: https://github.com/nyithukha/CMPE311/tree/main/Project2